Sam Thorpe
Michelle Hoang
Thy Nguyen

A Brief Summary and Bifurcation Analysis of the Lorentz Model of Atmospheric Circulation

Sam Thorpe
Michelle Hoang
Thy Nguyen

Our goals:

The goal of our project is to use the fourth order derivative Runge-Kutta method to numerically integrate the Lorenz equations over a range of parameters. We also want to investigate the behavior of the system as it evolves towards chaos. We give a short summary of the history of the model and plot time series solutions as well as phase portraits for various choices of parameters.

Our Roles:

Sam Thorpe: I'm in charge of researching the project materials. I checked out The Nature and Theory of the General Circulation of the Atmosphere written by Edward N. Lorenz at the Science Library. Lastly, I made graphs and phase portraits of the solutions and the code.

Michelle Hoang: I'm in charge of writing the introduction and help with the coding of Lorenz equations. I looked up *Deterministic Nonperiodic Flow* (Lorenz, 1963) on the internet.

Thy Nguyen: I'm in charge of writing the abstract and help with the coding of Lorenz equations. One of the resources that I used was the text book for Math 118, Nonlinear Dynamics and Chaos (Strogatz, 2000).

Lorenz Mathematical Model

Edward Norton Lorenz is an American mathematician and meteorologist, and a contributor to the chaos theory and inventor of the strange attractor notion, and he coined the term butterfly effect. Lorenz was born in West Haven, Connecticut, on May 23, 1917.

Sam Thorpe
Michelle Hoang
Thy Nguyen

He studied mathematics at both Dartmouth College in New Hampshire and Harvard University in Cambridge, Massachusetts. During the World War II, he served as a weather forecaster for the United States Army Air Corps. After his return from the war, he decided to study meteorology, in which he earned two degrees from the Massachusetts Institute of Technology, where he became a professor for many years. Lorenz has received many awards for his work from 1969 to recently in 2004.

Lorenz studied the way air moves around in the atmosphere, for which he built a mathematical model. The atmosphere is a fluid whose circulation possesses a highly complex structure. The circulation is governed by a set of laws which are known to a fair degree of precision, and in principle it should be possible to use these laws to deduce the nature of circulation in the atmosphere. As Lorenz studied weather patterns more closely, he began to realize that the weather did not always change as predicted. He found that unpredictable things could happen to make the weather turn out differently. Sometimes even a tiny change could end up making a huge difference. He explained this by using a butterfly as an example. He asserts that something as small as a butterfly flapping its wings in China could change the weather in the United States a few days later. This is possible because the butterfly moved a little bit of air that moved more air and so on until the moving air reached the other side of the word. He observed that minute variations in the initial values of variables in his primitive computer weather model would result in divergent weather patterns. Thus, this sensitive dependence on initial conditions came to be known as the butterfly effect.

Sam Thorpe
Michelle Hoang
Thy Nguyen

Lorenz noted that the foundation of any attempt to explain the circulation of the atmosphere should be to understand the physical laws which govern it. We begin our study of chaos with the equations which Lorenz himself studied, these are given by:

$$\begin{aligned} x' &= \sigma(y - x), \\ y' &= \mathrm{r}x - y - xz, \\ z' &= xy - \mathrm{b}z, \end{aligned}$$

where σ, r, b > 0 are parameters. These equations describe a greatly simplified model of convection rolls in the atmosphere, as well as models of lasers and dynamos, and finally these equations *exactly* describe the motion of a the chaotic waterwheel.

First we shall briefly discuss the free parameters of the model. σ is commonly known as the Prandtl number, which can be interpreted as a measure of the ratio of the fluid viscosity to the thermal conductivity of a substance, a low number indicating high convection. r is known as the Rayleigh number. It measures how hard we're driving the system, relative to the dissipation. In the context of the chaotic waterwheel, this ratio expresses a competition between the gravity and inflow, which tend to spin the wheel. The Rayleigh number appears in other parts of fluid mechanics, notably convection, in which a layer of fluid is heated from below. There it is proportional to the difference in temperature from bottom to top. For small temperature gradients, heat is conducted vertically but the fluid remains motionless. When the Rayleigh number increases past a critical value, an instability occurs. The hot fluid is less dense and thus begins to rise, and the cold fluid on top begins to sink. This sets up a pattern of convection rolls. As the Rayleigh number is further increased, the rolls become wavy and eventually chaotic.

Sam Thorpe
Michelle Hoang
Thy Nguyen

The neat mechanical model of the Lorenz equations was invented by Willem Malkus and Lou Howard as MIT in the 1970s. The simplest version is a toy waterwheel with leaky paper cups suspended from its rim.

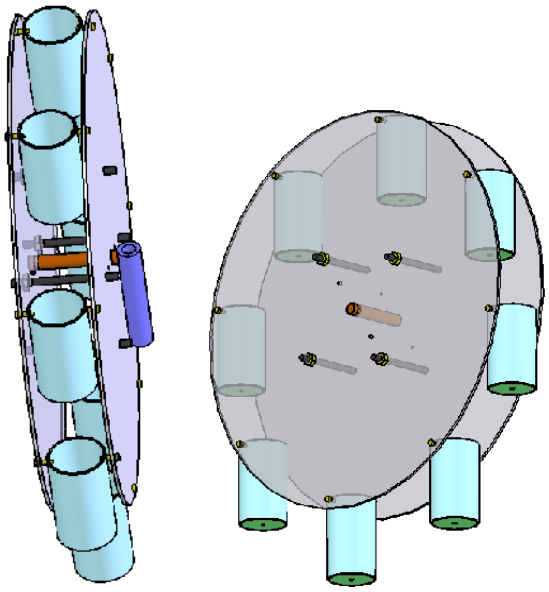

Figure1 . The chaotic waterwheel invented by Willem Malkus and Lou Howard at MIT. If the flow rate is to slow or leakage to great the wheel cannot move. As flow is increased the wheels periodic motions become chaotic and display sensitive dependence on initial conditions.

Water is poured in steadily from the top. If the flow rate is too slow the top cups never fill up enough to overcome friction, so the wheel remains motionless. If the flow rate is faster, the top cup gets heavy enough to start the wheel turning. Eventually the wheel settles into a steady rotation in one direction or the other. As the flow rate is increased past a critical value however, the cups become too heavy for the wheel to sustain true periodic motion and the rotation in either direction becomes unstable. After enough time, rotation in either direction becomes equally possible, and the outcome displays sensitive dependence on the initial conditions. MIT students have even been known to place bets on the direction of the spin after fixed amounts of time.

Sam Thorpe
Michelle Hoang
Thy Nguyen

Coding the Lorenz system

Attached is the C-code we have written which implements the fourth-order Runge-Kutta method of numerical integration to solve the Lorenz system of three first order differential equations. In order to assure that we get an accurate picture of the system's dynamics, we decided to solve the system out to 100 seconds, in increments of 0.1 seconds. The choice of the 0.1 second step is purely a practical matter, since we did all the plotting for this project in matlab, we needed to email large data files to one another, and another order of magnitude of resolution seemed unnecessary. In contrast, however, in coding the system in matlab itself, we chose a resolution of 0.01 seconds, since this choice had no practical consequences. As is demonstrated by our plots below, our O(4) Runge-Kutta code provided an excellent approximation, since the exact same dynamics were observed using both methods. All initial conditions were set at (0.1, 0.1, 0.1) near the origin.

Critical points

Recall our system:

$$x' = \sigma(y - x),$$
$$y' = rx - y - xz,$$
$$z' = xy - bz,$$

The system has critical points when $x' = y' = z' = 0$.

First note that the trivial solution $x = y = z = 0$ cleary works.

Now assume $\sigma, r, b > 0$.

Then $x' = 0 \Rightarrow \sigma(y - x) = 0 \Rightarrow y = x$.

Now $z' = 0 \Rightarrow xy - bz = 0 \Rightarrow xy = bz$,

then by $y = x$ we have $xy = bz \Rightarrow x^2 = bz \Leftrightarrow x = \sqrt{bz}$.

Sam Thorpe
Michelle Hoang
Thy Nguyen

$$\text{Now } y' = 0 \Rightarrow rx - y - xz = 0 \Rightarrow rx - y = xz,$$

$$\text{thus by } x = \sqrt{bz} \text{ we have } y = (r - z)\sqrt{bz},$$

$$\text{then } y = x \Rightarrow \sqrt{bz} = (r - z)\sqrt{bz} \Rightarrow 1 = (r - z) \Rightarrow z = r - 1.$$

Thus we have established the system has critical points at:

$$(0, 0, 0),\ (+\sqrt{b(r-1)},\ +\sqrt{b(r-1)},\ r-1),\ (-\sqrt{b(r-1)},\ -\sqrt{b(r-1)},\ r-1).$$

Immediately we see that $\forall r < 1$, we have only one critical point located at the origin. As $r \to 1$ we see that 2 new critical points emerge. This is the telltale sign of a pitchfork bifurcation, and indeed, that is precisely what occurs.

Manipulating the Rayleigh parameter, and a linearized analysis.

For the purposes of this project we have chosen to fix the σ and b parameters and vary the Rayleigh number (r). We chosen to let σ = 10, and b = 8/3 as these were the exact values used by Lorenz in the original study in which he discovered the butterfly effect. As we let r vary from ½ to 28 (in dimensionless units) we can observe structural changes in the form of the time series solutions. However, these changes are much more apparent in the phase space, thus we have decided to include both descriptions of the evolution of the system in our analyses. The purpose of this section is to use a linearization of the system to tract changes in eigenvalues which depend on r, and thus extract information about the systems dynamics as r is systematically varied.

Consider the Jacobian of the system:

$$J = \begin{bmatrix} -\sigma & \sigma & 0 \\ r - z & -1 & -x \\ y & x & -b \end{bmatrix}$$

At the origin, the characteristic equation is given by:

$$p_{\vec{0}}(\lambda) = -\lambda^3 - (b + 1 + \sigma)\lambda^2 - (2b + 1)\lambda - \sigma b(1 - r).$$

Sam Thorpe
Michelle Hoang
Thy Nguyen

Then taking the roots of the above polynomial we have:

$$\lambda_1 = -b, \quad \lambda_2, \lambda_3 = -\frac{1}{2}(\sigma+1) \mp \frac{1}{2}\sqrt{(\sigma+1)^2 + 4\sigma(r-1)},$$

Thus when $r < 1$, we have $\lambda_1, \lambda_2, \lambda_3 < 0$,

$\therefore$ the origin is a stable node.

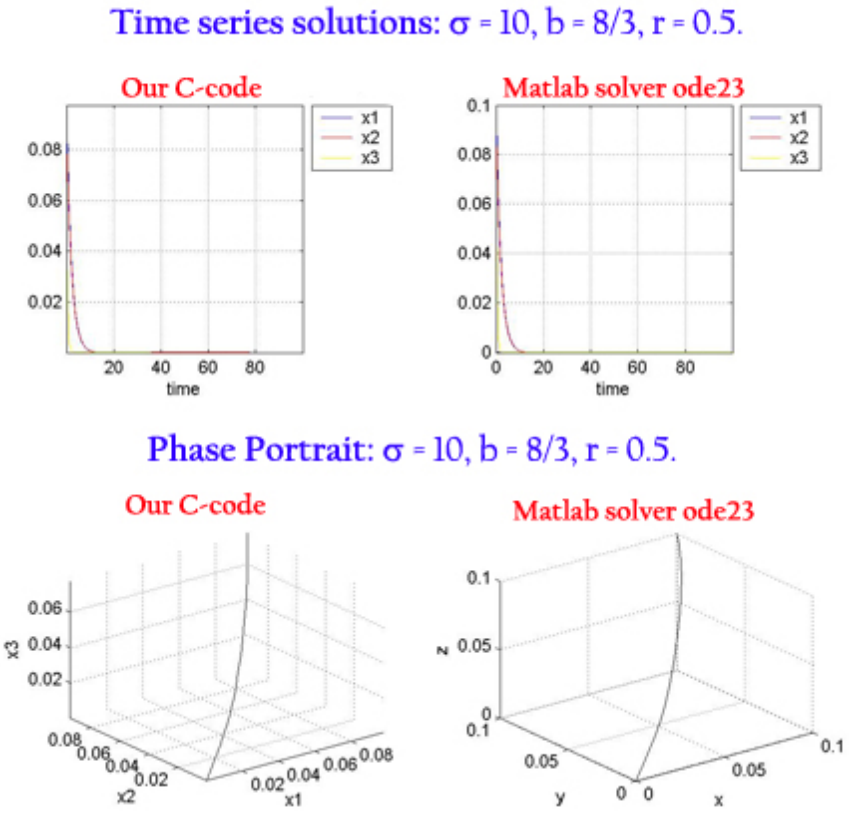

Figure 2. All trajectories, including the one pictured above, move towards the origin ,thus the origin is an asymptotically stable node.

Now for $r > 1$, the characteristic equation above gives:

$$\lambda_1, \lambda_3 < 0, \quad \lambda_2 > 0,$$

thus the origin becomes an unstable saddle point.

However, as we saw above, at $r = 1$ a pitchfork bifurcation

Sam Thorpe
Michelle Hoang
Thy Nguyen

gives rise to 2 new critical points. Then taking the Jacobian at these new points (which is the same by symmetry) we have characteristic equation:

$$p_{\vec{x}_c}(\lambda) = \lambda^3 + (b+1+\sigma)\lambda^2 + (b\sigma+br)\lambda + 2\sigma b(1-r),$$

then as $r \to 1^+$, we have $\lambda_1, \lambda_2, \lambda_3 < 0$,
and thus the emergent critical points are stable nodes.
However, for r larger than about 1.35,
we have $\lambda_1 < 0$, $\mathrm{Re}(\lambda_2), \mathrm{Re}(\lambda_3) < 0$,
and thus the emergent critical points become stable spirals.

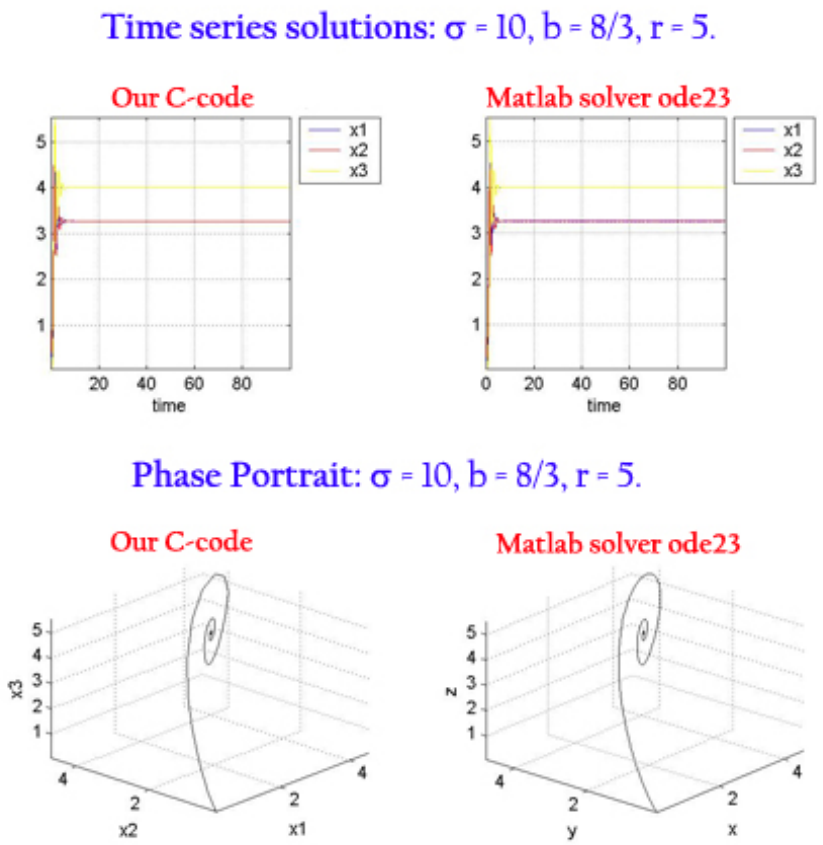

Figure 3. The origin has become unstable. Instead the phase space is divided into two basins of attraction corresponding to the two emergent spiral points.

Another interesting change in the phase portrait emerges as r is further increased. As $r \to r_0 \approx 13.926$, a *homoclinic* bifurcation

Sam Thorpe
Michelle Hoang
Thy Nguyen

takes place, in which an unstable limit cycle emerges.This fact cannot be derived directly from the linearized system, however it *is* evidenced in our numerical analysis by the fact that the chosen trajectory oscillates closely around a well defined curve before eventually spiraling towards the critical point. The shape of the boundary of this curve is indeed the shape of the emergent limit cycle.

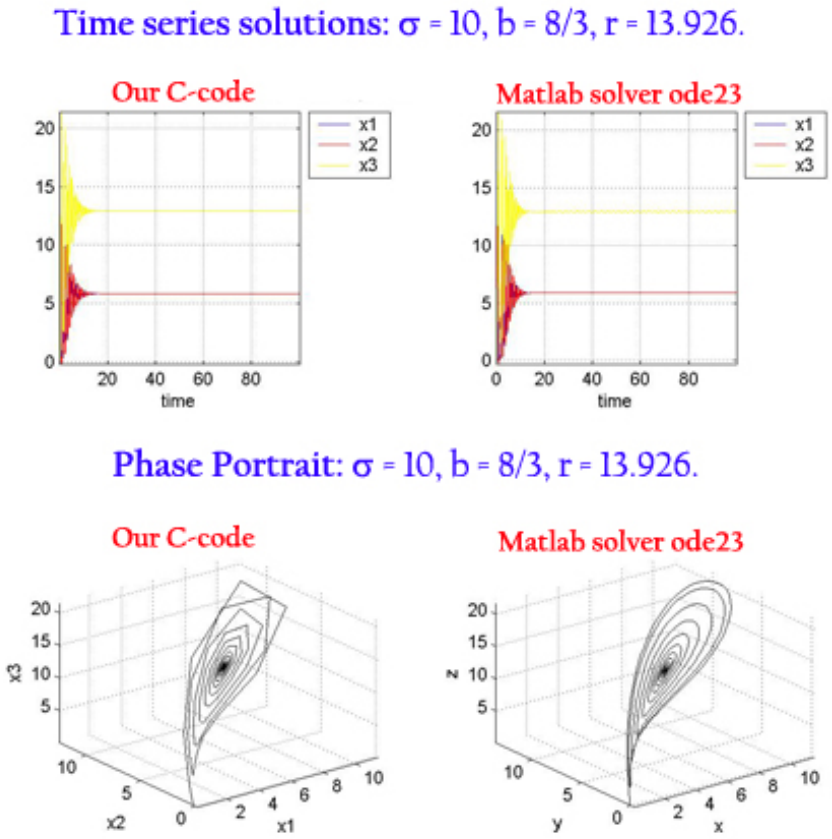

Figure 4. The emergent critical points remain stable spirals, however, a unique periodic solution, or limit cycle has also emerged. The limit cycle is unstable and trajectories perturbed slightly inside of it will spiral towards the center as shown above.

Now as r is further increased the emergent critical points do not remain

Sam Thorpe
Michelle Hoang
Thy Nguyen

stable. Referring back to the characteristic equation obtained above:

$$p_{\vec{x}_c}(\lambda) = \lambda^3 + (b+1+\sigma)\lambda^2 + (b\sigma+br)\lambda + 2\sigma b(1-r),$$

it can be shown by the Routh-Harwitz criterion that $\forall r > 1$,

$$\mathrm{Re}(\lambda_2), \mathrm{Re}(\lambda_3) > 0 \quad \text{when} \quad (\sigma - b - 1)r > \sigma(\sigma + b + 3).$$

Thus as $r \to r_H \equiv \dfrac{\sigma(\sigma+b+3)}{(\sigma-b-1)}$, we observe a subcritical Andronov-Hopf bifurcation in which the emergent critical points lose their stability to the unstable limit cycle which emerged at $r = r_0$.

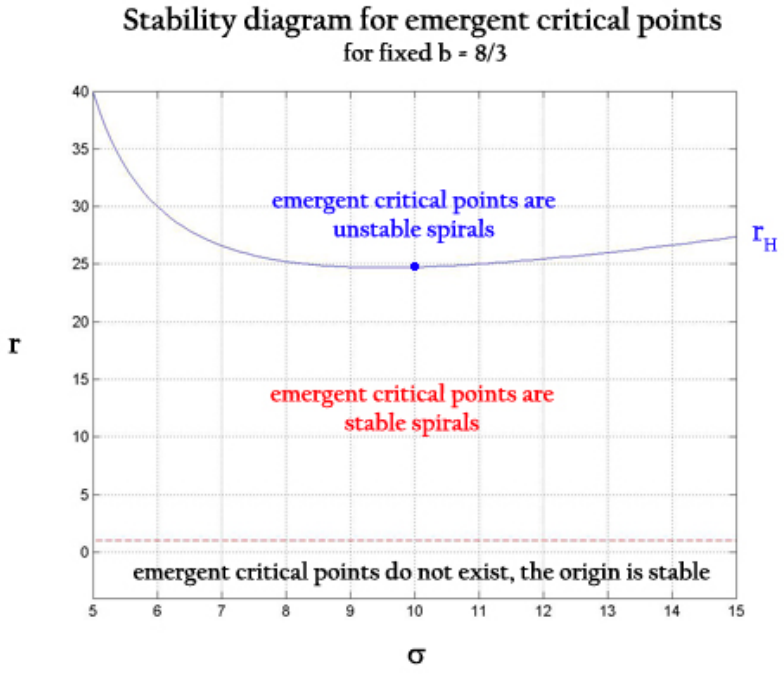

Figure 5. For fixed b, there exists a function $r_H(\sigma)$ *such that for any choice of sigma, If we choose* $r > r_H(\sigma)$ *then a subcritical Andronov-Hopf bifurcation occurs. Moreover, for each b, there exists a choice of* σ *such that the function* $r_H(\sigma)$ *tends to infinity and thus no bifurcation occurs. For our fixed b=8/3, this sigma is given by* $\sigma = 5/3$.

Sam Thorpe
Michelle Hoang
Thy Nguyen

As r is increased past $r_H \approx 24.74$, both emergent critical points are unstable, and the system's motion becomes chaotic. At r = 28

trajectories spiral away from both critical points, but jump unpredictably from oscillation about one to oscillation about the other.
Note that allthough these trajectories move away from both critical points, a simple argument based on the Lyapunouv function given by:

$$V_L(x,y,z) = rx^2 + \sigma y^2 + \sigma(z-2r)^2,$$

shows that trajectories near the emergent critical points cannot leave the ellipsoid defined by:

$$\frac{r}{b}x^2 + \frac{1}{b}y^2 + (z-r)^2 = r^2.$$

Thus we see that the trajectories forever move along what is known as a *strange attractor*. The Lorentz attractor or the "butterfly" attractor seen below was the first ever discovered, and is undeniably the most famous.

Sam Thorpe
Michelle Hoang
Thy Nguyen

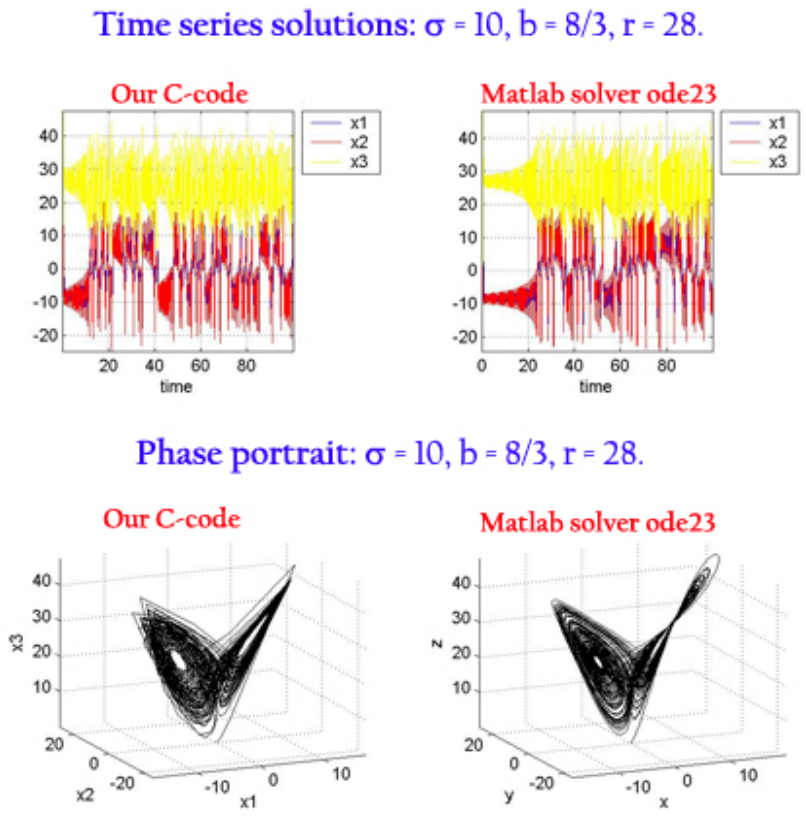

Figure 6. All trajectories are drawn to a zero volume surface known as a strange attractor. Trajectories switch unpredictably between oscillations away from the two unstable critical points.

References

Lorenz E., N. (1963) Deterministic nonperiodic flow. *Journal of Atmospheric Science*. 20, 130.

Lorenz E., N. *The Nature and Theory of the General Circulation of the Atmosphere*. World Meteorological Organization, 1967.

Strogatz, S. *Nonlinear Dynamics and Chaos: with Applications to Physics, Biology, Chemistry and Engineering.* Westview Press, 2000.

Sam Thorpe
Michelle Hoang
Thy Nguyen

C-Code: implementation of O(4) Runge-Kutta.

```
#include<stdio.h>
#include<math.h>
main()
{

float sigma,b,r;
double h;
double x1[10000],x2[10000],x3[10000],t[10000];
double f_1(double x1, double x2,double x3, double t, float sigma);
double f_2(double x1, double x2,double x3, double t, float r);
double f_3(double x1, double x2,double x3, double t, float b);

double dx1dt,dx2dt,dx3dt;
double k1_x1,k2_x1,k3_x1,k4_x1;
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

```
double k1_x2,k2_x2,k3_x2,k4_x2;
double k1_x3,k2_x3,k3_x3,k4_x3;

int i,N;

FILE *output;
output = fopen("final_data","w");

t[0] = 0.0;
x1[0] = 0.1;
x2[0] = 0.1;
x3[0] = 0.1;

h = 0.1;
N = 100.0/h;


printf("   \n");
printf("The purpose of the program is to solve the lorentz system using
o(4) Runge-Kutta\n");
printf("   \n");
printf("Please specify values for the three free parameters of the
system:\n");
printf("   \n");
printf("Sigma should be a positive real number, usually, but not
necessarily ranging from 0-20.\n");
printf(" let sigma = ");
scanf("%f", &sigma);
printf("   \n");
printf("b should be a positive real number, usually, but not
necessarily ranging from 0-5.\n");
printf(" let b = ");
scanf("%f", &b);
printf("   \n");
printf("r should be a positive real number, usually, but not
necessarily ranging from 0-30.\n");
printf(" let r = ");
scanf("%f", &r);
printf("   \n");
printf("Thank you\n");
printf("   \n");
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

```
for(i=1;i<=N;i++)
{

k1_x1 = h*f_1(x1[i-1],x2[i-1],x3[i-1],t[i-1], sigma);
k1_x2 = h*f_2(x1[i-1],x2[i-1],x3[i-1],t[i-1], r);
k1_x3 = h*f_3(x1[i-1],x2[i-1],x3[i-1],t[i-1], b);

k2_x1 = h*f_1(x1[i-1] + (k1_x1/2.0), x2[i-1] + (k1_x2/2.0), x3[i-1] +
(k1_x3/2.0), t[i-1] + (h/2.0), sigma);
k2_x2 = h*f_2(x1[i-1] + (k1_x1/2.0), x2[i-1] + (k1_x2/2.0), x3[i-1] +
(k1_x3/2.0), t[i-1] + (h/2.0), r);
k2_x3 = h*f_3(x1[i-1] + (k1_x1/2.0), x2[i-1] + (k1_x2/2.0), x3[i-1] +
(k1_x3/2.0), t[i-1] + (h/2.0), b);

k3_x1 = h*f_1(x1[i-1] + (k1_x1/2.0), x2[i-1] + (k2_x2/2.0), x3[i-1] +
(k2_x3/2.0), t[i-1] + (h/2.0), sigma);
k3_x2 = h*f_2(x1[i-1] + (k1_x1/2.0), x2[i-1] + (k2_x2/2.0), x3[i-1] +
(k2_x3/2.0), t[i-1] + (h/2.0), r);
k3_x3 = h*f_3(x1[i-1] + (k1_x1/2.0), x2[i-1] + (k2_x2/2.0), x3[i-1] +
(k2_x3/2.0), t[i-1] + (h/2.0), b);

k4_x1 = h*f_1(x1[i-1] + k3_x1, x2[i-1] + k3_x2, x3[i-1] + k3_x3, t[i-1]
+ h, sigma);
k4_x2 = h*f_2(x1[i-1] + k3_x1, x2[i-1] + k3_x2, x3[i-1] + k3_x3, t[i-1]
+ h, r);
k4_x3 = h*f_3(x1[i-1] + k3_x1, x2[i-1] + k3_x2, x3[i-1] + k3_x3, t[i-1]
+ h, b);

t[i] = t[0] + i*(h);
x1[i] = x1[i-1] + ((1.0)/(6.0))*(k1_x1 + 2.0*k2_x1 + 2.0*k3_x1 + k4_x1,
sigma);
x2[i] = x2[i-1] + ((1.0)/(6.0))*(k1_x2 + 2.0*k2_x2 + 2.0*k3_x2 + k4_x2,
r);
x3[i] = x3[i-1] + ((1.0)/(6.0))*(k1_x3 + 2.0*k2_x3 + 2.0*k3_x3 + k4_x3,
b);

printf("i = %i, t[i] = %10.5e, x1[i] = %10.5e, x2[i] = %10.5e, x3[i] =
%10.5e\n",i,t[i],x1[i],x2[i],x3[i]);
fprintf(output,"%i, %10.5e, %10.5e, %10.5e,
%10.5e\n",i,t[i],x1[i],x2[i],x3[i]);

}
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

```
printf("   \n");
fclose(output);

}

double f_1(double x1,double x2, double x3, double t, float sigma)
{
double dx1dt;

dx1dt = sigma*(x2 - x1);
return dx1dt;
}

double f_2(double x1,double x2, double x3, double t, float r)
{
double dx2dt;

dx2dt = (r*x1) - x2 - (x1*x3);
return dx2dt;
}

double f_3(double x1,double x2, double x3, double t, float b)
{
double dx3dt;

dx3dt = (x1*x2) - (b*x3);
return dx3dt;
}
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

Matlab Code: implementation of ode23 solver

```
clear;

%load final_1a_data.txt;    %r = 1/2
%load final_1b_data.txt;   %r = 5
%load final_1c_data.txt;   %r = 13.926
%load final_1e_data.txt;   %r = 28
%return;

time = final_1a_data(:,2);
x1 = final_1a_data(:,3);
x2 = final_1a_data(:,4);
x3 = final_1a_data(:,5);

samplingrate = 100;
samplestep = 1/samplingrate;
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

```
%solver parameters
tspan=[0:samplestep:99.9];
ICvector=[.1 .1 .1];
%NOTE: these are the same as used in our C-code

[t,y] = ode23(@tempfunklorentz,tspan,ICvector);
solution=y(:,1);

%lorentz parameters from tempfunklorentz
global sigma
global r
global b

%%%%%%%%%%%%PLOTS%%%%%%%%%%%%%%

figure  %TIME SERIES%
subplot(2,2,1),plot(time,x1,'b',time,x2,'r',time,x3,'y');
title(['time series solutions for lorentz oscillator with parameters sigma
=',num2str(sigma),',b = ',num2str(b),',r = ',num2str(r)]);
axis('tight');
legend('x1','x2','x3',-1);
grid('on');
xlabel('time');
subplot(2,2,2),plot(t,y(:,1),'b',t,y(:,2),'r',t,y(:,3),'y');
%title(['time series solutions for lorentz oscillator with parameters sigma
=',num2str(sigma),',b = ',num2str(b),',r = ',num2str(r)]);
axis('tight');
legend('x1','x2','x3',-1);
grid('on');
xlabel('time');

subplot(2,2,3),plot3(x1,x2,x3,'k');  %PHASE PORTRAIT%
title(['phase portrait for lorentz oscillator with parameters sigma =',num2str(sigma),',b =
',num2str(b),',r = ',num2str(r)]);
axis('tight');
grid('on');
xlabel('x1');
ylabel('x2');
zlabel('x3');
subplot(2,2,4),plot3(y(:,1),y(:,2),y(:,3),'k');
%title(['phase portrait for lorentz oscillator with parameters sigma =',num2str(sigma),',b =
',num2str(b),',r = ',num2str(r)]);
axis('tight');
grid('on');
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

```
xlabel('x');
ylabel('y');
zlabel('z');
```

Solver handle: tempfunklorentz

```
function dydt=tempfunklorentz(t,y);

%parameters
global sigma
sigma = 10;
global b
b = 8/3;
global r
r = 24.74;

y=y';
dydt=zeros(length(t),3);

dydt(:,1) = sigma*(y(:,2) - y(:,1));
dydt(:,2) = r*y(:,1) - y(:,2) - y(:,1).*y(:,3);
dydt(:,3) = y(:,1).*y(:,2) - b*y(:,3);
```

Sam Thorpe
Michelle Hoang
Thy Nguyen

```
dydt = dydt';
```